# ARKA-Lite · Guion

Versión sin demostración

**LÁMINA 1 · PRESENTA LUIS CARDOZE**

## Del endpoint al clúster

Buenos días. Ya vimos los fundamentos y una primera mirada a los temas que ahora vamos a desarrollar: cómo se guardan y se protegen los datos, cómo se comunican los servicios y cómo llevamos una aplicación del código a la operación.

Todo lo vamos a ver con ARKA-Lite, nuestro proyecto final del curso.

*Veamos cómo organizamos el recorrido.*

**LÁMINA 2 · PRESENTA LUIS CARDOZE**

## Qué vamos a ver

Primero les contamos lo que nos quedó de los fundamentos y la arquitectura, porque esos temas ya los vieron en la presentación anterior. Dedicaremos más tiempo a otros tres temas: la base de datos, cómo se comunican los servicios sin perder información y DevOps, que abarca cómo construimos, publicamos y operamos el sistema.

Después vemos la observabilidad, que es cómo sabemos lo que pasa adentro, y cerramos con los resultados.

*Empecemos con lo que nos quedó de los fundamentos.*

**LÁMINA 3 · PRESENTA LUIS CARDOZE**

## Lo primero: analizar antes de programar

De los fundamentos, lo primero que nos quedó es analizar antes de programar. Antes de escribir código nos hicimos tres preguntas sobre el negocio.

Primero, qué tiene que pasar. Una persona registra una solicitud, como una incidencia o un pedido de cambio, la envía, y otra persona la aprueba o la rechaza. De ahí salieron cuatro estados: borrador, enviada, aprobada y rechazada.

Segundo, quién hace cada paso. Quien solicita envía, el sistema avisa y otra persona decide. Gestionar la solicitud y producir el aviso son trabajos distintos, y por eso los separamos en dos servicios.

Tercero, qué no se puede permitir. Estas son las reglas de ARKA-Lite: una solicitud solo puede enviarse cuando está en borrador y una sola vez. Solo una solicitud enviada puede aprobarse o rechazarse. Cada regla quedó con su prueba automática.

*Con el qué ya definido, pasamos al cómo: escribir código que se pueda cambiar con confianza.*

LÁMINA 4 · PRESENTA LUIS CARDOZE

## Código mantenible: buenas prácticas

Otra cosa que nos quedó es cómo escribir código fácil de mantener. Son seis prácticas que sirven en cualquier lenguaje, ya sea Java, C# o PL/SQL.

La primera es que cada pieza hace una sola tarea: la que recibe el pedido no decide, y la que decide no guarda. La segunda es conectar las piezas desde afuera, para poder cambiarlas o probarlas por separado. La tercera es dejar las reglas del negocio en un solo lugar, sin repetirlas en la pantalla y en la base.

La cuarta es que todos los errores sigan el mismo formato y expliquen qué pasó. La quinta es usar los nombres del negocio, para que el código se lea como el proceso. Y la sexta es que cada regla tenga su prueba automática.

Es como una cocina ordenada, donde cada persona tiene su puesto y cada cosa su lugar. Así, cuando cambia una regla, tocamos un solo lugar y sabemos enseguida si algo dejó de funcionar.

*Esto es lo que nos llevamos a cualquier proyecto.*

LÁMINA 5 · PRESENTA LUIS CARDOZE

## Fundamentos: lo que nos quedó

Nos quedamos con tres ideas. Primero se analiza y después se programa, porque tener claras las reglas desde el inicio evita rehacer trabajo.

Segundo, separamos responsabilidades para que un cambio de regla toque un solo lugar y una prueba nos avise si algo se rompió. Tercero, las reglas del negocio no deben depender de la herramienta. Así siguen siendo las mismas si cambia la base de datos o la tecnología de la aplicación.

Estas tres prácticas se pueden aplicar en los sistemas actuales, sin importar el lenguaje ni cómo esté organizada la aplicación.

*Ahora le paso la palabra a Ricardo, que nos muestra la arquitectura del sistema.*

LÁMINA 6 · PRESENTA RICARDO FÁBREGA

## El sistema de un vistazo

Gracias, Luis. ARKA-Lite no es una aplicación grande, sino varias piezas pequeñas, cada una con su responsabilidad. Las vemos por capas, de arriba hacia abajo.

Arriba está la pantalla que usa la persona. No guarda información, solo les pasa los pedidos a los servicios.

En el centro están los dos microservicios, es decir, dos aplicaciones pequeñas con una función cada una. Uno maneja las solicitudes y sus estados, y el otro produce los avisos.

Debajo, cada servicio tiene su propia base de datos, y ninguno entra en la base del otro. Al final está la plataforma, donde todo corre, se prueba y se revisa.

Es como un edificio donde cada piso tiene su función, y por eso se puede cambiar uno sin reconstruir los demás.

*Con el mapa claro, veamos cómo colaboran esas piezas en la práctica.*

LÁMINA 7 · PRESENTA RICARDO FÁBREGA

## Cómo funciona en la práctica

Sigamos los números. La persona usa la pantalla, y la pantalla le pide al servicio de solicitudes que cree o envíe una solicitud. El servicio la guarda en su propia base.

Cuando la solicitud se envía, el servicio produce un mensaje pequeño para avisarle a notificaciones. A ese mensaje lo llamamos evento. Notificaciones crea el aviso y anota que ya recibió ese evento, y la pantalla consulta los avisos para mostrarlos.

Ninguna flecha une las dos bases. Es como dos departamentos que se intercambian notas, mientras cada uno mantiene el control de sus propios archivos.

*Ya vimos cómo se conectan por fuera. Ahora veamos cómo está organizado cada servicio por dentro.*

LÁMINA 8 · PRESENTA RICARDO FÁBREGA

## Por dentro de cada servicio: hexagonal

Cada servicio usa arquitectura hexagonal. El equipo anterior ya la explicó, así que les contamos solo lo que nos quedó.

Las reglas del negocio van en el centro, y todo lo demás se conecta alrededor, como un enchufe. Por un lado entran los pedidos de la pantalla, y por el otro se conectan la base de datos y el servicio de notificaciones. Así, las reglas no dependen de los detalles de esas conexiones.

Donde más nos sirvió fue en las pruebas. Cambiamos la conexión con la base por una versión en memoria, y con eso probamos todas las reglas en segundos, sin levantar una base.

Y esto solo ordena el código por dentro. Desde afuera, el servicio se instala y se usa como cualquier otro.

*Esto es lo que nos quedó de la arquitectura.*

LÁMINA 9 · PRESENTA RICARDO FÁBREGA

## Arquitectura: lo que nos quedó

Nos quedamos con cuatro ideas. Cada servicio es dueño de sus datos, y entre servicios viajan mensajes, no tablas.

La arquitectura hexagonal ordena el código por dentro y no cambia cómo se instala.

Y separar tiene un costo: más comunicación por red, más bases y más partes que pueden fallar. Por eso conviene usar microservicios cuando el negocio lo justifica. Para un sistema pequeño, una sola aplicación bien organizada puede ser la mejor decisión.

*Ahora entraremos en la base de datos. Le paso la palabra a Marlene.*

LÁMINA 10 · PRESENTA MARLENE BONILLA

## Dos bases, una por servicio

Gracias, Ricardo. El sistema tiene dos bases de datos, una para cada servicio, y ambas utilizan PostgreSQL, versión diecisiete.

Usamos dos porque cada servicio es dueño de sus datos. Solicitudes guarda los trámites y los eventos pendientes, mientras notificaciones registra los eventos que ya recibió.

Así, si una base cambia su estructura, la otra no se ve afectada. No hacemos consultas que combinen tablas de ambas bases ni definimos llaves foráneas entre ellas, es decir, referencias que obliguen a que un dato exista en la otra base.

Cuando un servicio necesita un dato del otro, lo pide por la red. Además, cada base funciona por separado y tiene su propio disco.

Por eso los datos permanecen cuando un servicio se reinicia y, si una base presenta una falla, la otra puede continuar trabajando.

*Ahora veamos qué información guarda cada una.*

LÁMINA 11 · PRESENTA MARLENE BONILLA

## La estructura de las bases

Veamos qué hay dentro. En la base de solicitudes, la tabla solicitud guarda el número, el tipo y el estado del trámite.

La propia base solo acepta los cuatro estados válidos, incluso si alguien intenta escribir directamente en la tabla. Al lado está la tabla evento, que funciona como una bandeja de salida.

Cada fila registra un evento, con un número único y una marca que indica si está pendiente o ya fue entregado.

La base de notificaciones tiene una tabla principal con los eventos que ya recibió. Funciona

como el libro de registro de la entrada de una oficina.

No existe una conexión directa entre las bases. Lo que las relaciona es el número del evento, que viaja en el mensaje entre los servicios.

*Ahora veremos un cambio importante frente a muchas aplicaciones actuales: dónde vive la regla del negocio.*

**LÁMINA 12 · PRESENTA MARLENE BONILLA**

## Del procedimiento almacenado al ORM

Aquí está uno de los cambios principales frente a muchas aplicaciones conocidas. La diferencia no es la base de datos, sino el lugar donde vive la regla del negocio.

En el esquema habitual, la persona pulsa Enviar, la aplicación llama a un procedimiento almacenado y ese procedimiento decide y guarda. Por lo tanto, la regla vive en la base.

En ARKA-Lite, la regla vive en el servicio junto con su prueba. Cuando la regla permite la operación, una herramienta llamada ORM prepara las instrucciones para guardar los datos. Esas instrucciones se escriben en SQL, el lenguaje con el que consultamos y modificamos la información de la base.

Esto no significa que los procedimientos estén mal ni que la base pierda importancia: sigue cuidando las llaves, las restricciones y los índices. Solo movimos la decisión a otro lugar.

Jossett lo mostrará con un ejemplo y después explicará qué es un ORM.

*Jossett, por favor.*

**LÁMINA 13 · PRESENTA JOSSETT FRANCESCHI**

## La misma regla, escrita de las dos formas

Gracias, Marlene. Veamos esa diferencia con una de las reglas que mencionó Luis: una solicitud solo puede enviarse cuando está en borrador.

A la izquierda, el procedimiento consulta el estado, valida la condición y actualiza el dato. No necesitamos leer cada línea para notar que la regla y el dato están juntos en la base.

A la derecha aparece la misma regla en el código de la aplicación. Si el estado no es borrador, se produce un error. Si lo es, la solicitud pasa a enviada.

La regla no cambia; cambia dónde se ejecuta. Y al tenerla en el código, podemos probarla en milésimas de segundo, sin una base.

*Pero si la regla vive en el código, alguien tiene que llevar el resultado a la base. Esa pieza es el ORM.*

LÁMINA 14 · PRESENTA JOSSETT FRANCESCHI

## ¿Qué es un ORM?

Acabamos de ver que la regla vive en el código, pero el dato se sigue guardando en la base. Y el código y la base no hablan el mismo idioma: para unirlos usamos un ORM.

ORM significa mapeo objeto-relacional. En palabras sencillas, es una herramienta que hace de intérprete entre la aplicación y la base de datos.

La aplicación trabaja con objetos, como una solicitud que tiene número, tipo y estado. La base, en cambio, trabaja con tablas, filas y columnas.

El ORM traduce de un lado a otro. Cuando la aplicación le pide guardar una solicitud, identifica la tabla y la columna de cada dato, genera el SQL y lo envía. Es como un intérprete en una reunión, donde cada parte habla su propio idioma.

En nuestro proyecto usamos Hibernate, aunque la idea existe en otras tecnologías; por ejemplo, .NET tiene Entity Framework.

El ORM no diseña la base por nosotros: las llaves, las relaciones y los índices los seguimos definiendo. Y si una consulta es compleja, podemos escribirla a mano.

*Veamos ahora cómo sabe el ORM qué parte de la clase va con qué parte de la tabla.*

LÁMINA 15 · PRESENTA JOSSETT FRANCESCHI

## El ORM por dentro: el mapa entre la clase y la tabla

Aquí vemos la traducción lado a lado. A la izquierda está la clase SolicitudEntity, que es como el código representa una solicitud. A la derecha, la tabla solicitud, donde la guardamos.

Las palabras en amarillo, las que empiezan con arroba, son etiquetas y forman el mapa. Table le indica al ORM que esta clase va en la tabla solicitud, e Id marca cuál es la llave primaria.

Los demás campos se emparejan por su nombre: tipo con tipo y estado con estado.

Abajo está lo que pasa por detrás. Cuando el código cambia el estado de una solicitud, el ORM usa ese mapa y arma este UPDATE. Los valores viajan como parámetros, en los signos de interrogación, igual que en un procedimiento.

Así no escribimos a mano el SQL de todos los días, y la base sigue recibiendo SQL normal, que podemos revisar.

*Para llegar a la base, el ORM necesita su dirección, usuario y clave. Veamos dónde guardamos esos datos.*

LÁMINA 16 · PRESENTA JOSSETT FRANCESCHI

## ¿Dónde guardamos los datos de conexión?

Toda aplicación necesita conocer la ubicación de la base, el usuario y la clave para entrar. A este conjunto de datos lo llamamos cadena de conexión.

En nuestros sistemas actuales la guardamos en un archivo web.config dentro del servidor. Esa ya es una buena práctica, porque la información no está escrita en el código.

ARKA-Lite aplica la misma idea con otra herramienta. En lugar de usar un archivo junto a la aplicación, recibe variables de entorno cuando arranca.

Así, el ambiente donde se ejecuta le entrega la dirección, el usuario y la clave. La misma aplicación puede funcionar en desarrollo, pruebas y producción sin cambiar su código.

Cada ambiente proporciona sus propios valores. Por eso, si cambia el servidor o renovamos una clave, solo actualizamos la configuración del ambiente.

*Ahora Marlene explicará cómo una transacción garantiza que los datos queden completos. Marlene, adelante.*

LÁMINA 17 · PRESENTA MARLENE BONILLA

## Una transacción: todo o nada

Gracias, Jossett. Este concepto es conocido para quienes trabajan con bases de datos, aunque aquí lo controlamos desde la aplicación.

Cuando enviamos una solicitud, abrimos una transacción: un grupo de cambios que se guardan juntos o se deshacen completos. Ese inicio corresponde al BEGIN. Primero cambiamos el estado a enviada y, dentro de la misma transacción, registramos el evento pendiente.

Al final, el COMMIT guarda ambos cambios juntos. Si algo falla en el proceso, hacemos ROLLBACK y deshacemos todo.

Funciona como una transferencia bancaria. El dinero debe salir de una cuenta y entrar en la otra, o no debe ocurrir ningún cambio.

En nuestro caso, el cambio a enviada y el evento pendiente quedan guardados juntos. La transacción evita que se conserve uno sin el otro. Esa garantía sostiene la interconexión que veremos a continuación.

*Antes de pasar a esa interconexión, resumamos lo que aprendimos sobre la base de datos.*

LÁMINA 18 · PRESENTA MARLENE BONILLA

## Base de datos: lo que aprendimos

De la base de datos nos llevamos cuatro ideas. El ORM genera el SQL rutinario, pero no elimina el trabajo de diseñar llaves, restricciones e índices.

También mantenemos los datos de conexión fuera del código y los entregamos desde el ambiente. La transacción continúa siendo una de las herramientas más confiables para proteger una operación.

Además, guardamos y controlamos la estructura de la base junto con el código. Para quienes trabajamos con bases de datos, el mensaje es claro: ese conocimiento sigue siendo fundamental.

*Ahora veremos cómo los servicios se comunican sin perder ni duplicar información. Le paso la palabra a Abdiel.*

LÁMINA 19 · PRESENTA ABDIEL BARRIA

## Síncrono o asíncrono

Gracias, Marlene. Como los servicios no comparten una base, necesitamos definir cómo se comunican, y hay dos formas de hacerlo. La síncrona se parece a una llamada telefónica, porque hacemos el pedido y esperamos la respuesta.

La usamos para crear, aprobar o rechazar una solicitud, ya que la persona necesita conocer el resultado de inmediato. Si el otro lado está caído, verá el error.

La comunicación asíncrona se parece a un correo. Enviamos el mensaje y seguimos trabajando sin esperar, aunque la respuesta pueda tardar unos segundos.

En nuestro sistema, si el otro servicio está caído, el mensaje queda guardado hasta que podamos entregarlo. En un pago con tarjeta, por ejemplo, la aprobación es síncrona porque necesitamos saber si fue aceptado.

En cambio, el comprobante por correo es asíncrono porque puede llegar después. La regla es sencilla. Usamos comunicación síncrona cuando necesitamos conocer el resultado para continuar, y asíncrona cuando el trabajo puede completarse después.

*Ahora veamos cómo hacemos que ese mensaje espere sin perderse mediante el patrón Outbox.*

LÁMINA 20 · PRESENTA ABDIEL BARRIA

## Patrón Outbox: guardar primero, avisar después

Si al enviar una solicitud llamáramos directamente a notificaciones y el servicio estuviera caído, la solicitud podría quedar enviada sin que nadie recibiera el aviso.

Para resolverlo usamos el patrón Outbox, que significa bandeja de salida. Dentro de una sola

transacción guardamos la solicitud y un evento pendiente, que es el registro de lo que falta comunicar.

La persona recibe la respuesta de inmediato, mientras el evento queda esperando como una carta que todavía no ha salido.

Después, un proceso llamado relevador revisa la bandeja cada cinco segundos y entrega los eventos pendientes. Podemos verlo como el mensajero del sistema.

Si la entrega funciona, marca el evento como entregado. Si falla, lo deja pendiente y vuelve a intentarlo en la siguiente revisión.

Así, aunque notificaciones se detenga, ningún evento se pierde y todos se entregan cuando regresa.

La misma idea puede aplicarse con procedimientos almacenados: guardamos el evento en una tabla de pendientes y una tarea programada se encarga de entregarlo.

*Como el sistema reintenta las entregas, ahora debemos evitar que el mismo evento produzca dos avisos.*

**LÁMINA 21 · PRESENTA ABDIEL BARRIA**

## Idempotencia: el mismo evento, un solo aviso

Un evento puede llegar dos veces. Por ejemplo, notificaciones lo recibe correctamente, pero la confirmación se pierde en la red. Como solicitudes no sabe que llegó, vuelve a enviarlo.

Para manejarlo usamos la idempotencia, una palabra técnica para una idea sencilla. Significa que repetir la misma operación no cambia el resultado.

Cada evento trae un número fijo, que llamamos el número del evento. Cuando llega, notificaciones lo busca en la lista. Si es nuevo, crea el aviso y registra el número. Si ya estaba registrado, lo ignora.

Es como un portero con una lista de invitados, porque quien ya entró no puede entrar una segunda vez.

Las dos soluciones se complementan. La bandeja de salida evita perder eventos y la idempotencia impide duplicar avisos.

Este principio sirve para cualquier proceso que tenga reintentos, como un pago, un correo o una importación de archivos.

*Con ambas garantías claras, veamos en una sola imagen quién habla con quién, y cómo.*

LÁMINA 22 · PRESENTA ABDIEL BARRIA

## Resumen: quién habla con quién, y cómo

Con las piezas claras, veamos el recorrido completo en una sola imagen. Es el resumen de esta parte.

Arriba está lo que ocurre mientras la persona espera. El navegador solo conoce la pantalla, y la pantalla pasa cada pedido por un intermediario que lo lleva al servicio de solicitudes. Es como la recepción de una empresa: pedimos un departamento y nos conectan con él.

Solicitudes aplica la regla, guarda la solicitud y su evento en la misma transacción y responde de inmediato. Esa es la parte síncrona.

Abajo está lo que pasa después, sin que nadie espere. El relevador toma el evento de la bandeja de salida y se lo entrega a notificaciones, que revisa el número, crea un solo aviso y anota que ya lo recibió. Esa es la parte asíncrona.

Dos detalles unen todo. Las piezas se encuentran por su nombre y no por una dirección fija, y cada llamada lleva un número de seguimiento, distinto del número del evento, que nos permite reconstruir el recorrido en los registros. Moises lo retomará en la observabilidad.

*Con el recorrido completo a la vista, resumamos lo aprendido sobre la interconexión.*

LÁMINA 23 · PRESENTA ABDIEL BARRIA

## Interconexión: lo que aprendimos

De la interconexión nos llevamos cuatro ideas. Esperamos la respuesta cuando la necesitamos para continuar y dejamos para después el trabajo que puede esperar.

También aprendimos que guardar primero y avisar después es más confiable que coordinar dos sistemas al mismo tiempo.

Cuando una operación se reintenta, el receptor debe reconocer lo repetido. Además, los servicios solo comparten el formato del mensaje, así que cualquier cambio debe coordinarse en ambos lados.

Estas decisiones no dependen de Java y pueden aplicarse a los sistemas que ya tenemos.

*Ahora veremos cómo llevamos el código y los datos hasta un servidor. Le paso la palabra a Kevin, de infraestructura.*

LÁMINA 24 · PRESENTA KEVIN

## ¿Qué es DevOps? Del código a la operación

Gracias, Abdiel. DevOps no es una herramienta ni un puesto específico. Es una forma de trabajar en la que desarrollo y operación colaboran y cada cambio sigue un camino repetible.

Podemos imaginarlo como una línea de producción. Cada producto pasa por las mismas

estaciones, en el mismo orden, y ninguna revisión se omite.

Primero escribimos y revisamos el cambio. Después lo probamos, lo empaquetamos, lo publicamos, lo instalamos y lo operamos en el servidor.

También vigilamos el sistema mediante sus registros. Lo que aprendemos durante la operación vuelve al código y el ciclo comienza otra vez.

En esta parte, yo explicaré Docker, Jossett mostrará el camino automático de los cambios y Moises explicará Kubernetes.

*Comencemos por entender cómo se complementan Docker y Kubernetes.*

**LÁMINA 25 · PRESENTA KEVIN**

## Docker y Kubernetes: no compiten, se complementan

Docker y Kubernetes se escuchan juntos con frecuencia, y a veces se presentan como alternativas. En realidad cumplen funciones distintas y se complementan.

Docker empaqueta la aplicación con todo lo necesario para ejecutarla, dentro de una imagen que funciona igual en una computadora o en un servidor. Así evitamos que algo funcione en una máquina y falle en otra.

Kubernetes comienza donde termina Docker: decide dónde corren esas aplicaciones, las vigila y reemplaza las que fallan.

Pensemos en el transporte de carga que conocemos en Panamá. Docker es el contenedor estándar, que se mueve igual sin importar lo que lleva, y Kubernetes es el puerto que los acomoda, los vigila y reemplaza el que se daña.

*Ahora veamos cómo una receta termina convertida en un contenedor.*

**LÁMINA 26 · PRESENTA KEVIN**

## De código a contenedor

Un contenedor nace a partir de tres conceptos. Todo comienza con una receta llamada Dockerfile, donde indicamos qué se instala y qué archivos se copian.

Con esa receta construimos una imagen, que funciona como un molde. Contiene la aplicación y únicamente lo necesario para ejecutarla.

La imagen no cambia, así que instalamos exactamente la misma que probamos. Cuando ponemos esa imagen a funcionar, obtenemos un contenedor.

Del mismo molde pueden salir varios contenedores iguales. Además, el nuestro se ejecuta con un usuario sin privilegios, por seguridad: si alguien abusara de la aplicación, tendría muy poco margen para hacer daño.

Por eso podemos reemplazar un contenedor y crear otro a partir de la misma imagen, sin reconstruir la aplicación.

*Con la receta, el molde y el contenedor claros, veamos qué ventajas nos ofrece Docker.*

**LÁMINA 27 · PRESENTA KEVIN**

## Ventajas de Docker

Docker nos ofrece varias ventajas. Primero, la misma imagen funciona en la computadora, en pruebas y en producción.

También es liviano, porque un contenedor arranca en segundos y no necesita cargar un sistema operativo completo como una máquina virtual.

Además, cada aplicación mantiene aisladas sus dependencias. Una puede usar Java 26 y otra Node en el mismo servidor sin interferir entre sí.

Podemos limitar la memoria y la capacidad de procesamiento, o CPU, que utiliza cada contenedor. Así, una aplicación que se descontrola no consume todos los recursos ni afecta a las demás.

Las imágenes también tienen versiones, de modo que podemos volver atrás utilizando una anterior. Finalmente, levantamos el sistema con un comando en lugar de seguir una lista manual.

*Ese comando no levanta una sola pieza, sino el sistema completo.*

**LÁMINA 28 · PRESENTA KEVIN**

## Todo el sistema con un comando

Un solo contenedor no representa todo el sistema. Con Docker Compose levantamos mediante un comando los dos servicios, las dos bases, la pantalla y las herramientas de registros.

En total se ejecutan ocho contenedores. Compose los conecta dentro de una red privada, donde cada uno encuentra a los demás por su nombre, como mencionó Abdiel.

También los inicia en orden. Primero levanta las bases y después cada servicio espera hasta que su base esté disponible.

Además, entrega la configuración desde afuera, incluidos los datos de conexión que explicó Jossett.

El beneficio es directo. Una persona nueva puede levantar el sistema con un comando, sin configurar cada componente por separado.

*Con el sistema reproducible, veamos cómo un cambio llega hasta una imagen publicada. Le paso la palabra a Jossett.*

LÁMINA 29 · PRESENTA JOSSETT FRANCESCHI

## Git: el historial y main

Gracias, Kevin. Antes de hablar de CI/CD necesitamos dos ideas de Git, la herramienta con la que guardamos el código.

La primera es el historial. Git guarda cada avance como un punto en una línea de tiempo. Así sabemos qué cambió, quién lo hizo y cuándo. También podemos volver a un punto anterior.

Esa línea de tiempo principal y estable se llama main. Es la versión oficial, la que se instala.

Es como el historial de versiones de un documento compartido.

*Ahora, si todos trabajáramos directamente sobre main, nos estorbaríamos. Para eso existen las ramas.*

LÁMINA 30 · PRESENTA JOSSETT FRANCESCHI

## Las ramas: trabajar en paralelo sin tocar main

La segunda idea es la rama. En main nadie trabaja directamente, porque es la versión estable y tiene que seguir funcionando.

Cuando vamos a atender una funcionalidad o una corrección, creamos una rama. La rama nace de un punto de main: es una copia de main tal como está en ese momento. Ahí guardamos los avances sin tocar la versión oficial.

Arriba está la rama de una funcionalidad y abajo la de otra. Las dos avanzan al mismo tiempo, cada una por su lado, sin pisarse.

Cuando el trabajo está listo, la rama se integra a main. A eso le llamamos merge, y es la flecha verde.

Es como trabajar en una copia de un documento y, al final, integrar los cambios al original.

*Con esas dos ideas, veamos cómo un cambio viaja de una rama hasta main.*

LÁMINA 31 · PRESENTA JOSSETT FRANCESCHI

## Cómo entra un cambio, paso por paso

Juntemos las dos ideas: así entra un cambio a main.

Creamos una rama desde main, hacemos el cambio y, si hace falta, sus pruebas. Después hacemos push, que es subir la rama al repositorio compartido, y pedimos integrarla.

Otra persona revisa el cambio y, cuando lo aprueba, hacemos el merge. Así nada entra a la versión oficial sin revisión.

*Hasta aquí, todo lo hacemos las personas. Pero hay revisiones que se repiten en cada cambio, y esas las puede hacer una máquina. Eso es CI/CD.*

LÁMINA 32 · PRESENTA JOSSETT FRANCESCHI

## CI/CD: el control de calidad automático

Cada vez que llega un cambio repetimos las mismas revisiones: compilar, probar y publicar. CI/CD es dejar que una máquina las haga sola, siempre igual. Para eso armamos un pipeline: una lista de pasos que se ejecuta sola cuando ocurre algo en el repositorio.

Arriba está el primer camino: cada vez que hacemos push o pedimos integrar un cambio. El pipeline compila, es decir, arma la aplicación a partir del código, y corre todas las pruebas. El resultado queda en verde o en rojo; si queda en rojo, el cambio no entra. Esa es la integración continua, o CI.

Abajo, el segundo camino: cuando el cambio ya entró a main. El pipeline compila y prueba otra vez y, si todo pasa, publica la imagen que nos explicó Kevin, el paquete listo para instalar. Esa es la entrega continua, o CD.

Es como el control de calidad de una fábrica: cada producto pasa por las mismas revisiones. En nuestro proyecto, la instalación final todavía se hace con un comando.

*¿Dónde le decimos al pipeline qué hacer? En un archivo de texto. Veamos cómo luce.*

LÁMINA 33 · PRESENTA JOSSETT FRANCESCHI

## El pipeline se escribe como código

El pipeline no se configura con clics: lo escribimos en un archivo de texto que guardamos en el repositorio, junto al código. A la izquierda está el nuestro, simplificado. No hace falta leer cada línea: lo importante es que especifica tres cosas.

Primero, cuándo se ejecuta: ante un evento, como pedir integrar o llegar a main. Es la parte de arriba del archivo.

Segundo, qué hace: los pasos, en orden. En la siguiente lámina vemos cuáles.

Tercero, cómo debe hacerlo: en qué máquina y con qué herramientas, por ejemplo con una base de datos temporal para las pruebas, y con qué condiciones, como publicar la imagen solo cuando el cambio llega a main.

Abajo ven algunas herramientas que lo usan: cambia la forma de escribirlo, pero todas especifican lo mismo. Y como el archivo es parte del proyecto, también lo revisamos antes de que entre.

*¿Y qué pasos podemos poner en ese archivo? Veamos algunos ejemplos.*

LÁMINA 34 · PRESENTA JOSSETT FRANCESCHI

## Qué se puede automatizar

En ese archivo podemos poner los pasos que necesitemos. Estos son algunos ejemplos; los verdes son los que ya hace ARKA-Lite.

Lo básico es compilar, para confirmar que el código arma sin errores, y correr las pruebas unitarias, que validan que las reglas siguen funcionando.

También verificamos que la aplicación arranca con su base y publicamos la imagen lista para instalar.

La fila de abajo son ideas para crecer: revisar la calidad del código, buscar fallas de seguridad, instalar en un ambiente de pruebas y avisar al equipo cuando algo falla.

Cada paso que automatizamos deja de depender de que alguien se acuerde, y se hace igual cada vez.

*Veamos ahora qué pasa cuando una prueba detecta un problema.*

LÁMINA 35 · PRESENTA JOSSETT FRANCESCHI

## Cuando un cambio rompe algo

Veamos qué pasa cuando un cambio rompe una regla. Supongamos que alguien permite por error enviar dos veces la misma solicitud.

Arriba, sin revisión automática: el cambio se integra, nadie lo nota, se instala días después y, semanas más tarde, llega el reclamo de una persona usuaria. Para entonces, el daño ya está hecho.

Abajo, con el pipeline: en pocos minutos falla la prueba que protege esa regla y queda en rojo. El cambio no entra.

La persona corrige el problema, el pipeline vuelve a correr y, solo cuando todo está en verde, el cambio entra a main.

Por eso cada regla necesita una prueba: es la alarma que se activa cuando alguien modifica el comportamiento esperado.

*Con esto cerramos el recorrido de un cambio: del código a una imagen lista para instalar. Ahora veamos dónde se ejecuta. Le paso la palabra a Moises, que nos explicará Kubernetes.*

LÁMINA 36 · PRESENTA MOISES

## Cómo trabaja Kubernetes

Gracias, Jossett. Kubernetes requiere una forma distinta de pensar. No le damos instrucciones paso a paso, sino que describimos el estado que queremos mantener.

Funciona como un termostato. No le indicamos cuándo debe encender y apagar el aire, sino

la temperatura que queremos, y el equipo hace los ajustes.

En Kubernetes escribimos en un archivo qué imagen debe usar, cuántas copias necesitamos y qué configuración corresponde.

Luego entregamos ese archivo mediante un comando y Kubernetes crea los elementos solicitados.

A partir de ahí, compara continuamente lo que existe con lo que pedimos. Si falta algo, lo crea y, si sobra, lo elimina.

Por eso el sistema puede repararse solo. Como el archivo se guarda con el código, la configuración tampoco depende de la memoria de una persona.

*Ahora veamos dos conceptos nuevos. Un Pod es el espacio que Kubernetes le da a cada contenedor, y un Service es el nombre fijo para encontrarlo.*

**LÁMINA 37 · PRESENTA MOISES**

## Contenedores y Pods: cómo se complementan

Sigamos el diagrama de izquierda a derecha. La imagen es el molde que hizo Docker, y el Deployment es el pedido que le hacemos a Kubernetes: en este ejemplo, dos copias de esa imagen.

A la derecha está un servidor del clúster, que es el grupo de servidores donde corren las aplicaciones. Cada contenedor vive dentro de un Pod, que le da su dirección, su memoria y su CPU. Siguiendo con el puerto, el contenedor es la carga y el Pod, su espacio en el patio.

Arriba está el Service. Como los Pods se reemplazan y cambian de dirección, el Service mantiene un nombre fijo y reparte el trabajo entre las copias.

En ARKA-Lite usamos una copia de cada pieza. Para tener dos, basta con cambiar un número en su archivo.

*Ya conocemos las piezas principales. Ahora veamos qué capacidades nos ofrece Kubernetes con ellas.*

**LÁMINA 38 · PRESENTA MOISES**

## Las bondades de Kubernetes

Kubernetes aporta capacidades que Docker por sí solo no ofrece. Si un Pod se cae, levanta otro. Si aumenta la carga, pedimos más copias cambiando un número, y el Service reparte el trabajo entre ellas.

También actualiza poco a poco, sin apagar todo, y permite volver a la versión anterior si algo falla.

Controla la memoria y la CPU de cada Pod con una reserva y un tope. Es como un

presupuesto: la reserva es lo que apartamos al planificar y el tope es el máximo permitido. En ARKA-Lite lo aplicamos a la pantalla.

Por último, guarda las claves aparte, en un objeto llamado Secret, y los datos en un disco que sobrevive al reinicio del Pod. Todo queda escrito en archivos, sin depender de la memoria de una persona.

*Veamos qué ocurre cuando una de esas copias se cae.*

**LÁMINA 39 · PRESENTA MOISES**

## Cuando un Pod se cae

Supongamos que el servicio de solicitudes se cae por un error o por el reinicio del servidor. Kubernetes detecta que falta una copia y crea otra con la misma imagen.

La nueva copia espera hasta que su base esté lista. Después pasa la revisión de salud y comienza a recibir trabajo otra vez, sin que una persona tenga que intervenir.

Como en el laboratorio mantenemos una sola copia, tenemos unos segundos sin servicio. Con dos copias, la interrupción casi no se notaría.

Mientras tanto, los eventos pendientes permanecen guardados en la base y no se pierde información.

Aquí se conectan tres partes de la presentación. La base guarda, la bandeja de salida espera y Kubernetes repara.

*Con este ejemplo cerramos la explicación de Kubernetes y resumimos lo aprendido de DevOps.*

**LÁMINA 40 · PRESENTA MOISES**

## DevOps: lo que aprendimos

De DevOps nos llevamos varias ideas. Docker y Kubernetes se complementan: uno empaqueta la aplicación y el otro la ejecuta y la cuida.

La imagen es el entregable que construimos una vez y reutilizamos entre ambientes. Además, la configuración pertenece al ambiente y no al código.

También aprendimos a describir el estado que queremos y a comprobar que los servicios realmente arrancan, en lugar de verificar solamente que compilan.

El mayor valor es que todo quedó escrito. Tenemos la receta de cada imagen, el pipeline y los archivos de Kubernetes, sin depender de la memoria de una persona.

*El sistema ya está instalado. Ahora veamos cómo sabemos qué ocurre dentro.*

LÁMINA 41 · PRESENTA MOISES

## Observabilidad: tres preguntas

Observabilidad es el nombre que damos a la capacidad de entender qué ocurre dentro del sistema sin tener que adivinar.

La resumimos mediante tres preguntas. La primera es si el sistema está vivo. Cada servicio informa si funciona y si puede comunicarse con su base.

La segunda pregunta es qué pasó. Para responderla utilizamos los registros, también llamados logs, que funcionan como la bitácora de cada servicio.

La tercera es por dónde pasó un pedido. Cada llamada lleva un número de seguimiento, como el número de guía de un paquete. No es el número del evento que vimos con Abdiel: aquel evita avisos duplicados y este nos permite seguir el recorrido.

Ese número permite saber por dónde pasó, a qué hora y en qué punto se detuvo. Sin él tendríamos que revisar cada servidor por separado.

Con observabilidad, buscamos un solo número y reconstruimos todo el recorrido.

*Veamos cómo aplicamos estas ideas a un caso concreto, cuando un aviso no llega.*

LÁMINA 42 · PRESENTA MOISES

## Un aviso no llegó: ¿cómo lo encontramos?

Supongamos que alguien envió su solicitud, pero no recibió el aviso. No queremos revisar los registros de cada servicio uno por uno.

Por eso los reunimos en un solo lugar con tres herramientas. Promtail es el recolector: toma los registros de cada contenedor. Loki es la base de datos donde se guardan. Y Grafana es la pantalla donde los consultamos.

Escribimos el código de la solicitud y aparecen juntos los registros de ambos servicios, en orden: cuándo se envió, cuándo se entregó el evento y si notificaciones creó el aviso. Si el evento siguiera en la bandeja de salida, veríamos dónde se detuvo.

Un servicio encendido no siempre funciona bien. Por eso también revisamos resultados, como la cantidad de eventos pendientes.

*Con este caso resuelto, resumamos lo que aprendimos sobre observabilidad.*

LÁMINA 43 · PRESENTA MOISES

## Observabilidad: lo que aprendimos

De observabilidad nos llevamos tres ideas. Los registros deben organizarse desde el primer día, porque hacerlo más adelante cuesta mucho más.

En un sistema con varios servicios necesitamos un número que acompañe cada pedido de

principio a fin. También debemos distinguir entre estar encendido y funcionar correctamente.

Por eso no revisamos únicamente la salud del proceso, sino también sus resultados. Si tuviéramos que comenzar por una sola práctica, usaríamos un número por operación en todos los registros.

*Para cerrar, le paso la palabra a Abdiel, con los resultados.*

LÁMINA 44 · PRESENTA ABDIEL BARRIA

## Resultados para el negocio

Gracias, Moises. Lo que les contamos se traduce en estos resultados para el negocio.

Primero, los eventos pendientes permanecen guardados si se interrumpe la entrega, y reenviar el mismo evento no crea otro aviso.

Segundo, el trámite no se detiene. Aunque el servicio de avisos esté caído, la persona puede enviar su solicitud y el aviso llega cuando el servicio regresa.

Tercero, las reglas del negocio se cumplen y cada una tiene una prueba que la vigila.

Cuarto, Kubernetes puede reemplazar una copia que falla, mientras los eventos pendientes permanecen guardados en la base.

Por último, los cambios llegan con menos riesgo, porque detectamos los errores en minutos y antes de que afecten a los usuarios.

*Estos resultados no dependen de la suerte, sino de varias decisiones que trabajan juntas.*

LÁMINA 45 · PRESENTA ABDIEL BARRIA

## Lo que sostiene los resultados

Detrás de cada resultado hay una práctica que aprendimos y aplicamos. La transacción, la bandeja de salida y el control de duplicados mantienen consistentes los datos.

La configuración está separada del código, así que podemos cambiar las claves sin modificar la aplicación.

También ejecutamos el mismo sistema en distintos ambientes mediante un solo comando. Las pruebas automáticas revisan cada cambio antes de incorporarlo.

Además, reunimos los registros en un solo lugar y relacionamos cada versión con el cambio que la produjo.

La mayoría de estas prácticas puede aplicarse sin reemplazar la tecnología que ya utilizamos.

*Si tuviéramos que resumir todo el recorrido en tres ideas, serían las siguientes.*

LÁMINA 46 · PRESENTA ABDIEL BARRIA

## Lo que nos llevamos

De toda la capacitación nos llevamos tres ideas. La primera es diseñar pensando en la falla, porque algún componente fallará y debemos definir cómo responderá el sistema.

La segunda es automatizar las revisiones y dejar la infraestructura descrita en archivos. Así podemos hacer cambios con mayor confianza y menor riesgo.

La tercera es hacer visible el sistema. No podemos diagnosticar ni explicar con claridad lo que no vemos.

*Ese es el aprendizaje que conecta todo el recorrido: prepararnos para las fallas, revisar los cambios y poder entender qué ocurre en el sistema.*

LÁMINA 47 · PRESENTA ABDIEL BARRIA

## Gracias

Comenzamos con una sola operación en Java y terminamos con un sistema completo de dos servicios, cada uno con su propia base de datos.

El sistema quedó empaquetado, instalado, vigilado y documentado. Además, puede conservar la información y recuperarse cuando una de sus partes falla.

Muchas gracias. Con gusto respondemos sus preguntas.